

ME/CS134 Final Report

Project: Tangrams

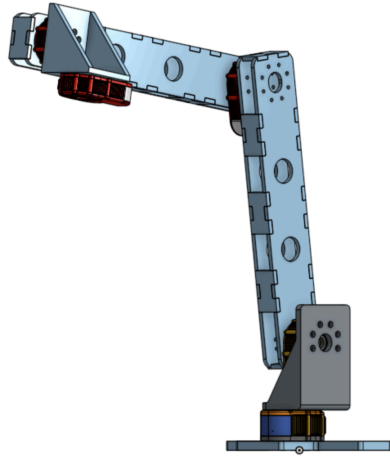
Emily Baylock, Ji Hyun Jeon, Catherine Zheng, Yunha Jo

Overview

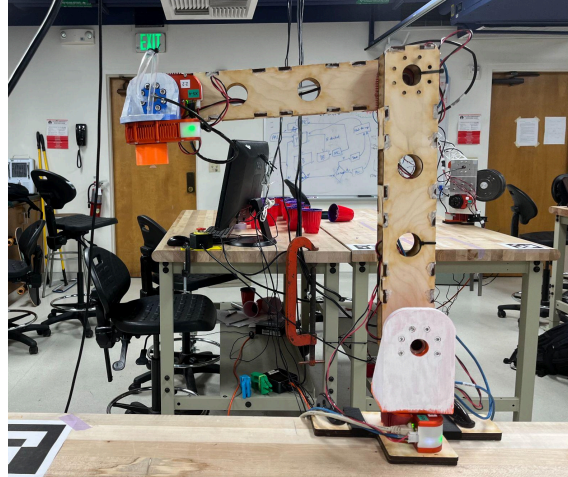
We built a robot that could recreate a tangram configuration given a picture. The robot arm is expected to first, identify individual pieces and their center points on the picture, then transform the coordinates into the real world coordinates, detect where the tangram pieces are in real life, and finally pick up the pieces in order and place them at their desired positions. The robot will also correct the pieces if they are at wrong locations or in wrong orientations, and it will continue building a tangram configuration even if some pieces are not detected. The user can interact with the robot in different ways, such as moving the correctly placed pieces into another position. The human can also build tangrams with the robot: if the human signals to the robot that it wants to move things around, the robot will wait for the human to stop signaling its interference, then it will correct the piece if it is incorrectly placed.

Mechanism and Hardware

We used 5 DOF arm for the project, 3 DOFs coming from base, shoulder, and elbow motors and 2 DOFs coming from the wrist of the arm. First 3 DOFs allow the arm to reach most of (x, y, z) positions in the workplace. 1 DOF in the wrist was in the YZ-plane so that the robot could keep the pieces parallel to the ground as it moved and to flip a piece, and the other DOF was in XY-plane so that the arm could rotate the piece to the correct orientation. We had two links that placed the motors approximately 40 cm apart each, so our total workspace was around 80cm x 80cm. The links were laser cut using wood, but to ensure that the contortion was minimized, for each link, we used two wood cutouts with supports between them. We had two L-brackets, one at the base and one at the wrist. The L-bracket at the base was designed with larger ribs to prevent it from flexing. On the other hand, the wrist L-bracket was designed to be smaller and lighter to minimize weight on the arm. We also designed a special connector piece that was attached at one of the wrist motors to connect the suction cup and tubing to the arm. Each of the pieces discussed in this section went through at least two rounds of manufacturing and testing before joining our final robot, as we predicted we would have enough trouble with the HEBI motors and precision of the arm that we wanted to make the other pieces as robust as possible in around two weeks.



CAD of the Arm



Actual Arm

Kinematics, Trajectory, and Continuity

Our task coordinate is the 5 dimensional coordinates of the tip of the arm, or the end of the suction cup. The task coordinates are made of (x, y, z, α, β) , with (x, y, z) representing the position, α representing the orientation about the z axis, and β representing the orientation about the x axis. We used the kinematic chain code from 133a for the mapping between the joint and task space. For inverse kinematics, we used a 5×5 dimension Jacobian matrix and a weighted inverse with a γ of 0.01 to ensure smooth processing of singularities.

We are using quintic task splines to generate trajectories and a segment queue to keep track of the splines and ensure continuous movement. We are using a modified version of the SegmentQueueDelayed code, with the task splines able to keep track of whether the vacuum pump should be on and whether the arm is ready for the next movement command, and the segment queue able to communicate that to the arm node. When the arm receives a Pose Array message of [Pickup position, Drop Off position], splines are added to the segment queue such that the arm goes to a desired position with a z offset, pushes down, and moves back up with appropriate vacuum controls. When the arm receives a Pose message, which only happens when a parallelogram needs to be flipped, splines are added such that the arm picks up the parallelogram, rotates it by the x axis, and drops it.

We also modified the task spline so that we can choose to add a waypoint to any spline. The waypoint is generated by looking at the midpoint between the initial position and final position. If the midpoint is too low (i.e. has a small z value) or if it is within 30 cm of the base of the arm, it is extrapolated until it is within an appropriate range. Thus, the waypoint ensures a smooth trajectory that doesn't go through a singularity. The waypoints were set to have the maximum

velocity, or 20 cm / sec, and the period of the spline is found through the equation of quintic splines, or $T = \frac{15 \Delta d}{8 V_{max}}$.

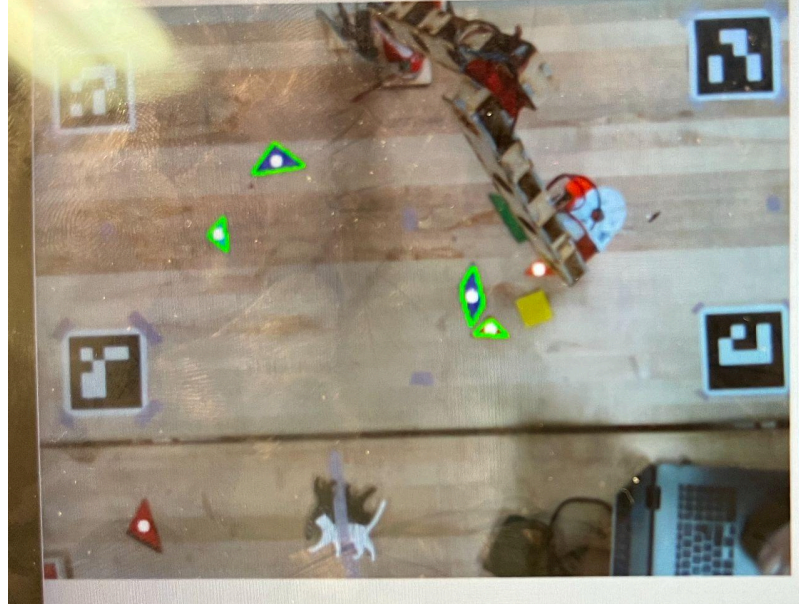
For gravity compensation, we are using the following equations:

$$\begin{aligned}\tau_{elbow} &= 3.5 \cos(\theta_2 + \theta_3) \\ \tau_{sholder} &= \tau_{elbow} + 3.5 \cos(\theta_2 + \pi/2)\end{aligned}$$

The gravity compensation along with the motor tuning helped align the actual position of the motors and the commanded position of the motors.

Camera and Detector

We are using the Realsense Depth camera for our detection. The detector is mainly used for detecting the tangram pieces in the workplace, and the pieces are determined by the shape, color, and the area. The detector creates different binaries based on different HSV color ranges, and it detects the pieces based on whether they are appropriate shape or size. We chose not to erode then dilate our image (we only dilate-erode), as this process began eroding away the sharp corners of our triangles. Crucially to detect our shapes we detect how many corners a polygon fit to the shape has, and thus cutting off the corners of the triangles occasionally led to the detector thinking they had more than three sides. In order to filter the pieces, we are using constant temporal filtering. We consider the piece to be present at a specific location if it has been detected in that location for a certain duration. This was done by creating an “exist” field for each piece, which was incremented every time the piece was detected where it was in the previous iteration, and decremented every time the piece was not detected or if it was far from the previously detected location. This temporal filtering was implemented to resolve some issues that we were facing with our preliminary filtering of averaging the piece’s position over a set number of iterations. In this case, noise (faulty readings) was filtered into the position, though it mitigated my choosing a large number of iterations. Another important issue that this aided in resolving was that the detector often failed to detect all of the pieces, meaning the arm constantly moved off to the side to get a better view of its workspace, resulting in inefficient workflow. The temporal filtering can handle the cases such as if the detector fails to detect the piece it detected in the previous iteration, if pieces are covered by the arm, if a piece is currently being moved by the arm, and when a piece has been moved to a different location or orientation. With existence values in place, even pieces that aren’t every tick are still considered as existing.



As seen in the image above, the pieces and their center points are being correctly detected. Note that centroids are calculated as pixels, which we first undistort from our camera lens, and then map to the world frame with our ArUco tags. Pieces covered by the robot arm are not detected, but their locations will not be reset, and the detector will ensure that they are in proper positions once the arm moves out of the way.

Software Architecture

Our software architecture is mainly comprised out of four parts:

1. PictureDetector

The picture detector receives a computer generated picture of tangrams, then it detects pieces, their shapes, center points, and angles and returns them as a list. The picture detector also determines the order in which the pieces should be placed, which is decided according to the distance between the piece centroid to the centroid of the picture (average of all centroids of the pieces weighted with their area). Pieces with smaller distances are placed first.

Upon receiving a picture, the detector first dilates and erodes the photo to widen the borders. Then, it converts the image to grayscale and uses the cv2 library to filter out the contours. Since cv2 returns a list of points of the contours, we use those points to create vectors and use vector geometry to figure out the angle and orientation of the shapes, including the side of the rhombus. We then pick the top corner of the photo to be the origin and find the centroids in pixel space. Then, we normalize the length of the vector by multiplying it with the ratio of the square tangram's size in real life to the pixel length of its vector to convert, converting the points from pixel world and return the orientation and centroid of the contours (given by cv2).

2. TangramDetector and Brain Node

The tangram detector continuously detects the pieces in the workspace according to the detection algorithm described above. It then creates a list of pieces with their center points and orientations in the workspace. The detector node receives the position of the arm every half a second, and it ignores detected centroids that are covered by or near the arm.

The brain node gets the pieces, their desired locations and orientations, and the order in which they should be placed in from the picture detector. Note that, the player can choose to insert a fifth ArUco tag into the camera frame, corresponding to a new tangram to solve, which resets the desired positions of pieces and signals the arm to start building the new shape. For each piece, it then calculates offset position, as we create an “exploded view” of the tangram configuration where each piece is first moved along the vector from its centroid to the global centroid to an offset position. Once the detector is certain that it has gotten the correct location and orientation of the piece, it will send a pose or posearray message to the trajectory node in order to move the piece. Pose is sent when the desired piece is a parallelogram, and it is on the wrong flip side. The Posearray message contains two poses, one for the desired location and orientation of the piece, and the other for actual location and orientation of the piece. The brain will wait for the arm to send a “done” message before attempting to send pose or posearray again. When the detector/brain is satisfied with the placement of the piece at its offset location, it will attempt to move the piece to the desired location. It will continue to send pose/posearray messages until it is satisfied with all of the locations and if it cannot detect any more pieces. Brain will then move the arm to a wait position, which is out of the camera’s view, and it will wait until it notices any changes.

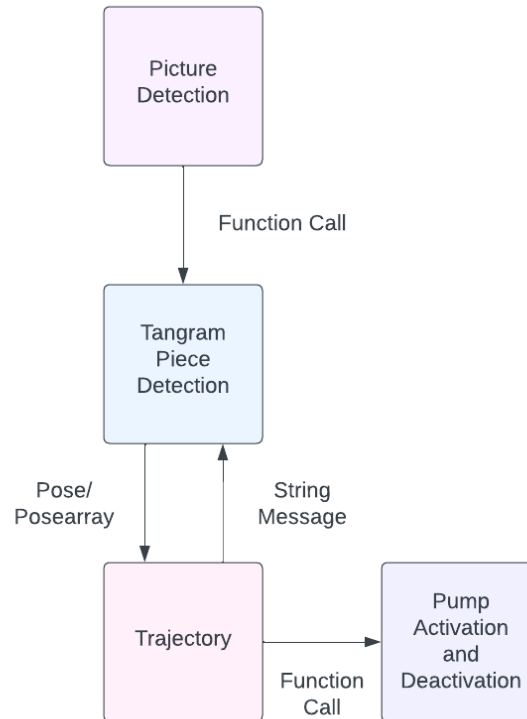
3. Trajectory Node

The trajectory node receives a pose/posearray message from the brain node. Once it receives the message, it extracts the location and orientation from the message and enqueue the necessary splines accordingly. If the message is a pose, which indicates that it is trying to move a parallelogram, the trajectory moves the arm into a flipping position and attempts to flip the piece by dropping it at an angle. If the trajectory has received a pose array, then the splines are enqueued such that the arm first moves to the actual location of the piece and then moves to the desired location through the waypoint. While the arm is moving, the trajectory node also turns the vacuum on and off. Every half a second, the trajectory sends the current position of the tip of the arm to the brain node through a point message, and once the motion is complete and every spline has been dequeued from the queue, it sends a “done” message to the brain node.

4. Vacuum Pump

Hardware wise, the vacuum is powered by a 10V power supply connected to a L298N motor driver and is controlled by an ArduinoMega through a serial connection. In software, the serial port connection is created during the node initialization and is closed during shutdown. The trajectory spline has a parameter that turns the vacuum on and off through a customized Vacuum class.

We're also using the stiffer vacuum tubing as the silicone one collapses too easily, and we punctured a hole in the tubing to keep the motor from stalling.



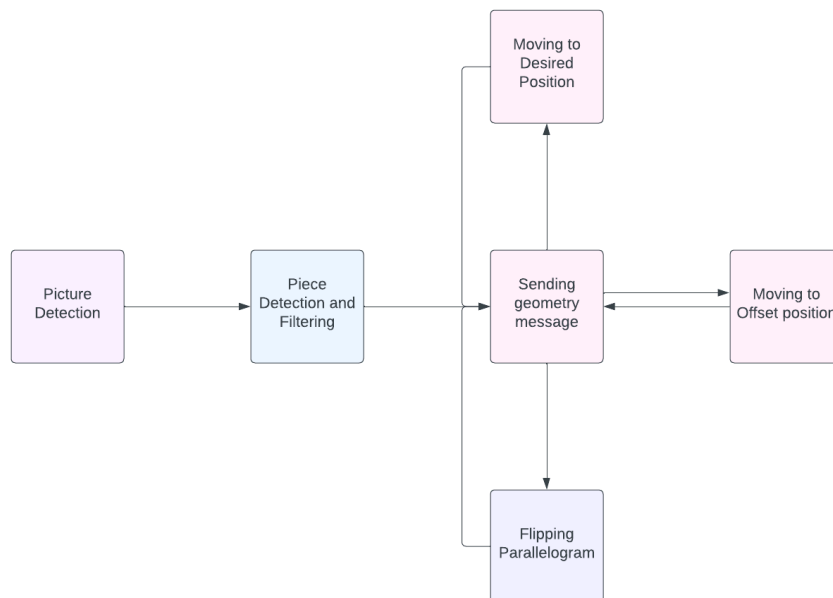
Behaviors and Failure Recoveries

The detector is able to receive two kinds of messages, a pose array that signifies where the pick up and place location and orientation of a piece is, and a pose that indicates where the parallelogram is. The pose arrays are used to move pieces from one place to another, whether it's some position on the table to offset or offset to goal position, and the pose is used to flip the parallelogram. Upon receiving the two different messages, the detector then sends a "received" string message to the detector that signals the detector to stop publishing the pose/pose arrays. When the detector has completed its actions, it then sends a "done" signal to the detector that triggers the publishing within the detector again.

The detector is capable of filtering pieces continuously while taking into consideration the location of the arm (so it doesn't detect itself!). This allows faster and more accuracy during

game play. It is also able to flip the parallelogram if it is on the wrong side. We also programmed the robot to first move pieces to an “offset” position before placing it in the desired position. The assumption here is that since the offset and desired positions are close to each other, the error between the two locations will likely be very similar, and so doing a pick and place from offset to goal would be much more accurate than doing straight from a farther original location to the goal.

The most commonly used recovery mode is of course detecting when pieces aren’t where they should be, and correcting their placement. If pieces are close to their exploded position, they will be placed directly to their desired position, but if they are moved far off from their exploded and desired positions, they will be placed back in the exploded positions before being put back in the desired position. We further ensure that the robot starts a few centimeters off the table as it goes to pick pieces up to account for stacked pieces, and it presses down more than strictly necessary in case a piece is leaning on another piece at an angle. This ensures that the suction cup will still close around the piece.



Shortcomings or Outstanding Items

We think the best way we could improve our system would be to improve our failure recovery modes. Some examples of improvements in our detector include the following. Currently, the detector isn’t always able to detect a piece partially covered by another, especially when the pieces have similar colors. Also, due to how we chose to filter our pieces, we didn’t track every piece at all times, meaning pieces sometimes don’t “exist” when determining the next piece to place. This could happen, for example, if another piece was placed on top of it and not removed

for a while. In this case, the order of the placement of pieces is thrown off, leading to several possible undesirable states, including pieces getting stacked on top of each other. As explained before, we do keep track of pieces for a while after they are seen, but we do eventually filter them out.

Another thing that could be improved is our human interaction component. Currently, a human hand has to be above one section of table for the robot to stop what it's doing. Being able to generalize it to the whole table (meaning the robot would recognize when someone reached into the workspace) would improve the whole system interaction component. Human interaction such as handing a robot a piece could have been cool as well. Also, when switching pictures, the robot doesn't clear out its current workspace, which means there's a high likelihood of the robot stacking pieces. So, implement a "clear workspace" whenever the desired photo changes would help with this problem. In general, the system's detection is still somewhat unreliable, with different shading causing problems with detection and filtering.

We also worked a lot on minimizing the robot's error when sending it to a position, but this was very hard to keep constant, as after running for a few hours, we often saw an offset in its positioning. This could be a problem in our URDF, a problem in how we take the zero position of our robot, or a combination of the two. While it was hard to troubleshoot, having a robot that can accurately pick up the pieces makes the finished tangrams so much more accurate, and really neatens up the project, so systematically checking that, and perhaps creating a more detailed error map than the linear x and y offsets we defined in the code could have elevated our project.

Real-World Connections

Over the term, we also enjoyed thinking about ways that our robot, or elements from our robot, could be useful in real-world robotics systems. That's not to say that no one wants a robot that can solve tangrams, but there are many cool, more practical uses for robots that use vacuum pumps. For example, Boston Dynamics' robot Stretch utilizes a much more powerful vacuum pump to sort and store boxes ([seen in the demo here](#)). As shown in the demo, the robot also detects what type of object it is picking up, and decides what to do with it. For example, it behaves differently when moving boxes as opposed to moving another, smaller robot (named Spot 😊). This is similar to how our robot made different choices in what to do with the different pieces it detected. Another cool feature of Stretch is the whole array of suction cups it has at the end of its arm, which could help pick up bigger items, or items without a perfectly flat surface. While all of our tangrams were flat, we considered possibly getting a bigger suction cup at one point when we were worried our tangrams were too heavy for the small one we originally got. Another interesting option would have been to use multiple small ones. Perhaps if they were connected to different pumps as well, they could have helped the robot pick up pieces if it was

slightly off-center, as only one suction cup out of several would need a perfect seal with the piece.

Another cool example of a robot that uses suction and similar detection are robots that sort recyclables! The AMP Cortex robot uses computer vision to first determine what materials the products coming down its conveyor belt are, and then uses a suction cup to remove ones that don't belong ([seen in the demo here](#)). This arm seems to be built with a closed loop linkage, though it is hard to fully see. Perhaps there are four bars connected from the suction cup to the ceiling, where the motors are? The idea of a closed loop linkage is interesting, though we probably don't have the capacity to build our robots on the ceilings! And their website states that they can achieve up to 80 picks per minute! Our robot was much, much slower than that. In a very similar demo of a very similar robot by Waste Robotics, we see one of these robots attempt to pick up a piece, and fail ([seen in the demo at the time step here](#), consider watching at 0.25 speed)! This failure seems to be due to either the robot not having enough suction to pick up the crushed can, or maybe the surface of the can wasn't flat enough for the suction cup to grip on to. Both of these were recovery modes that we considered for our robot as well, but luckily we didn't have too much trouble with either due to the pieces we are picking up.